

Instrumentation d'un miroir de boxe Combat Mirror

Rapport d'avancement n°1 : installation de RT-COSMIK et première chaîne de capture de mouvement

Aymeric Duchene

Polytech Montpellier, département MEA

en collaboration avec CSULB (Biomedical Engineering), Combat Mirror et le LAAS-CNRS (Gepetto)

27 septembre 2026

Abstract

Ce document retrace la mise en place du premier livrable du projet : l'installation de RT-COSMIK, la boîte à outils du LAAS qui estime en temps réel la cinématique articulaire d'une personne filmée par des caméras RGB. Il décrit l'environnement de départ, chaque problème rencontré et sa solution, la configuration de la caméra, les scripts développés, et une première évaluation quantitative sur une prise de boxe de 80 s. À la date du rapport, la chaîne complète (caméra → détection → squelette → modèle biomécanique → angles articulaires en CSV) fonctionne avec une seule webcam, à environ 15 à 22 résultats par seconde.

Contents

1	Contexte et objectifs	2
2	Point de départ et environnement	2
2.1	Première tentative sur MacBook	2
2.2	Environnement retenu	3
3	Architecture de RT-COSMIK	3
4	Chronologie de l'installation : problèmes et solutions	4
4.1	Moteur TensorRT de YOLO (26/09)	4
4.2	Mise en service de la webcam (26/09)	4
4.3	Premier squelette par IA (26/09)	4
4.4	Choix de la version de RT-COSMIK (27/09)	5
4.5	Calibration intrinsèque sans damier imprimé (27/09)	5
4.6	Premier lancement du pipeline complet (27/09)	6
4.7	Visualisation et synchronisation vidéo / CSV (27/09)	7
5	Scripts développés	7
6	Configuration de la caméra	8
7	Premiers résultats : prise demo_03	8
8	Limites actuelles et suite	10
A	Aide-mémoire : lancer une session	10
B	Modification apportée au code de RT-COSMIK	10

1 Contexte et objectifs

Le projet vise à instrumenter un miroir d'entraînement de boxe fabriqué par l'entreprise australienne Combat Mirror. Des capteurs de pression placés à l'arrière du miroir doivent mesurer l'intensité et la localisation des impacts. Ces données sont synchronisées avec la cinématique articulaire du boxeur, capturée par deux caméras RGB et le logiciel RT-COSMIK développé au LAAS (Toulouse). Une analyse par IA (par exemple un Transformer) doit ensuite évaluer la séance et produire un score.

Livrable	État au 27/09/2026
1 Installation de RT-COSMIK	Fonctionnel avec une webcam : chaîne complète jusqu'aux angles articulaires en CSV (objet de ce rapport).
2 Instrumentation du miroir	Non commencé.
3 Synchronisation et CSV	Côté caméra prêt : CSV horodatés par compteur d'images.
4 Algorithme de traitement (IA)	Non commencé, dépend du livrable 3.

Table 1: Avancement des livrables.

Ressources fournies par le LAAS. Le code de RT-COSMIK est public sur le GitHub de l'équipe Gepetto : <https://github.com/Gepetto/rt-cosmik>. La branche `offline-paper`, indiquée comme la plus à jour, a été fusionnée dans `main` le 22/09/2026 (commit 064353f). S'y ajoutent l'outil de calibration https://github.com/Gepetto/cams_calibration, un pont ROS 2 https://github.com/Gepetto/rtcosmik_ros, et un conteneur Docker de développement <https://github.com/MaximeSabbah/cosmik-dev-container>. Les caméras utilisées nativement par le LAAS sont des webcams Creative à bas coût.

2 Point de départ et environnement

2.1 Première tentative sur MacBook

La première tentative a consisté à cloner RT-COSMIK sur un MacBook Pro et à lancer `./scripts/bash/fetch_models.sh`, qui télécharge les poids des modèles.

Problème – Téléchargement des modèles impossible sur macOS

Symptôme. `ssl.SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED] unable to get local issuer certificate` lors de l'appel `urllib` du script.

Cause. Le Python 3.11 installé depuis `python.org` (`/Library/Frameworks/Python.framework`) n'utilise pas les certificats du système. Son paquet de certificats doit être installé séparément.

Solution. Exécuter `/Applications/Python 3.11/Install Certificates.command` (installe `certifi`), puis vérifier avec `urllib.request.urlopen('https://github.com')`.

RT-COSMIK s'appuie sur CUDA et TensorRT (moteurs de détection YOLO, modèle NLF sur GPU), ce qui exige un GPU NVIDIA. Le travail s'est donc poursuivi sur un PC Windows équipé d'une carte graphique NVIDIA, dans le conteneur Docker fourni par le LAAS.

2.2 Environnement retenu

Élément	Valeur
Hôte	Windows, Docker Desktop (moteur WSL 2), <code>usbipd</code> pour partager l'USB avec WSL
GPU	NVIDIA GeForce RTX 4070 SUPER, 12 Go, compute capability 8.9
Image Docker	<code>mmpose_image</code> , construite depuis le <code>Dockerfile</code> de <code>cosmik-dev-container</code>
Base de l'image	<code>nvidia/cuda:12.1.1-cudnn8-devel-ubuntu22.04</code> (Ubuntu 22.04, Python 3.10)
Bibliothèques	<code>torch 2.4.1+cu121</code> , <code>torchvision 0.19.1+cu121</code> , <code>TensorRT 11.3.0</code> , <code>NumPy 1.26.4</code> (épingle), <code>ROS 2 Humble</code> ; compilés depuis les sources : <code>Pinocchio 3.9.0</code> (avec <code>CasADi</code>), <code>CasADi 3.7.2</code> , <code>coal 3.0.2</code> , <code>eigenpy 3.12</code> , <code>FATROP</code> , <code>acados</code> , <code>example-robot-data</code>
RT-COSMIK	Préinstallé : <code>MaximeSabbah/RT-COSMIK</code> , branche <code>main</code> , commit <code>bd80b0d</code> (identique à la branche <code>nlf_humble</code> de <code>Gepetto</code>). Ajouté : <code>Gepetto/rt-cosmik</code> , branche <code>main</code> , commit <code>064353f</code>
Caméra	Webcam UGREEN (USB <code>0c45:2283</code>), une seule pour l'instant

Table 2: Environnement matériel et logiciel. Le relevé exact des versions est produit par `scripts/tools/collect_from_container.sh` (`docs/container_versions.txt`).

Le conteneur n'a pas de dossier partagé avec Windows : tous les fichiers de travail vivent à l'intérieur (`/root/workspace/ros_ws/`) et sont récupérés avec `docker cp`. Des instantanés ont été pris avec `docker commit` (`cosmik-backup:2026-09-26`, puis après la démonstration). Le fonctionnement de l'image et les moyens de la transporter sont décrits dans `docs/docker_export.md`.

3 Architecture de RT-COSMIK

La lecture du code de la branche `main` donne la chaîne de traitement suivante, pour chaque itération :

1. **Acquisition** : un processus par caméra lit le flux MJPEG de `/dev/video<i>` via `ffmpeg`, et non OpenCV, pour limiter la latence et enregistrer la vidéo sans ré-encodage. Une barrière synchronise les caméras.
2. **Détection** : YOLOv10n, sous forme de moteur TensorRT, localise la personne. Le moteur est construit avec une taille de lot fixe égale au nombre de caméras (`yolov10n_b<N>.engine`).
3. **Estimation de pose** : NLF (*Neural Localizer Fields*, modèle TorchScript `nlf_s_multi_0.2.2`) prédit, dans la boîte détectée, la position 2D/3D de 43 points choisis sur le maillage SMPL-X. Ces points jouent le rôle de marqueurs anatomiques virtuels (épine iliaque, C7, épaules, épicondyles, poignets, genoux, chevilles, pieds, tête).
4. **Reconstruction 3D** : triangulation entre caméras, ou, avec une seule caméra, estimation 3D monoculaire de NLF.
5. **Modèle biomécanique** : un modèle humain multicorps (`example-robot-data`, `Pinocchio`) est mis à l'échelle du sujet (taille, masse) sur les premières images, pendant lesquelles le sujet reste immobile.
6. **Cinématique inverse contrainte** : à chaque image, on cherche la configuration articulaire q qui minimise l'écart entre les marqueurs du modèle et les marqueurs mesurés, sous contraintes (limites articulaires, segments rigides). Deux solveurs existent : `sbs` (programme quadratique image par image, `quadprog`) et `mhe` (estimation sur horizon glissant, `acados` ou `FATROP`, à précompiler).

7. **Sorties** : `joint_angles.csv` (position et orientation du bassin, puis 36 angles articulaires en radians), `markers.csv` (43 marqueurs en 3D, en mètres), la vidéo `camera_<i>.mkv`, et une visualisation 3D Meshcat dans le navigateur.

4 Chronologie de l'installation : problèmes et solutions

4.1 Moteur TensorRT de YOLO (26/09)

Le moteur de détection a été généré avec `yolo export (.pt → ONNX → TensorRT 11.3)`, en lot de 2 (`yolov10n_b2_img640_fp32.engine`, 138 Mo) puis de 1 (`yolov10n.engine`, 91 Mo).

Problème – Le moteur généré est refusé par TensorRT

Symptôme. `trtexec --loadEngine=weights/yolo/yolov10n.engine` échoue : `Serialization assertion header.magicTag == kEXPECTED_MAGIC_TAG failed ... incompatible serialization version (1875 != 1953657958)`, alors que `trtexec` et le module Python sont tous deux en TensorRT 11.3.0.

Cause. L'examen des premiers octets (`xxd`) montre `53 07 00 00 7b 22 64 65 73 63...`, soit un entier de 4 octets (`0x753 = 1875`) suivi d'un JSON `{"description": "Ultralytics YOLOv10n model..."}`. Ultralytics préfixe le moteur TensorRT brut par ses propres métadonnées. TensorRT lit donc 1875 là où il attend sa signature `0x74727466` ("ftrt" = 1953657958).

Solution. Aucune conversion n'est nécessaire. RT-COSMIK charge le détecteur avec `ultralytics.YOLO(chemin, task="detect")`, qui sait lire ce format. Le script `scripts/tests/engine_metadata.py` lit ces métadonnées ; le moteur batch 1 est bien en batch 1, `imgsz [640, 640]`. **Leçon** : lire le code du chargeur avant de déboguer le format d'un fichier.

4.2 Mise en service de la webcam (26/09)

La webcam est partagée de Windows vers WSL 2 avec `usbipd attach --wsl --busid 5-2` et apparaît dans le conteneur comme `/dev/video0` (bus `usb-vhci_hcd.0-1`, c'est-à-dire USB/IP). Le premier test (`scripts/tests/test_webcam.py`) confirme la capture et la détection d'une personne par YOLO, mais à seulement **2,0 images/s** en `1280×720`.

Problème – Cadence de la caméra très faible

Symptôme. 2 fps mesurés avec OpenCV en `1280×720` pour une caméra annoncée à 30 fps.

Cause. Deux causes cumulées, isolées avec `v4l2-ctl --stream-mmap` :

- le réglage `exposure_dynamic_framerate` était à 1 (défaut constructeur : 0). La caméra baisse alors sa cadence pour allonger l'exposition dès que la lumière faiblit. Un test capuchon fermé l'a confirmé : les fps chutent puis remontent avec la lumière ;
- une fois ce réglage corrigé, la cadence reste plafonnée et *parfaitement stable* : 3,7 fps en `1280×720` et environ 13 fps en `640×480`. Diviser le nombre de pixels par 3 multiplie les fps par 3,5 : c'est une limite de bande passante du transport USB/IP de WSL, qui gère mal les transferts isochrones des webcams.

Solution. `v4l2-ctl -d /dev/video0 -c exposure_dynamic_framerate=0` à chaque branchement, et passage du pipeline en **640×480**. La solution définitive est un Linux natif (machine Debian de Polytech), où la caméra est vue directement, sans USB/IP.

4.3 Premier squelette par IA (26/09)

Le modèle NLF a été testé seul sur une photo prise par la webcam (script `test_skeleton_nlf.py`, méthode `detect_smpl_batched`, 24 articulations SMPL). La figure 1 montre le résultat, sujet en

garde. Le bassin, la colonne, les coudes, les poings et les jambes sont bien placés. Les épaules sont légèrement ramenées vers l'intérieur, car elles sont masquées par les bras en garde de face. Les pieds sont hors champ.

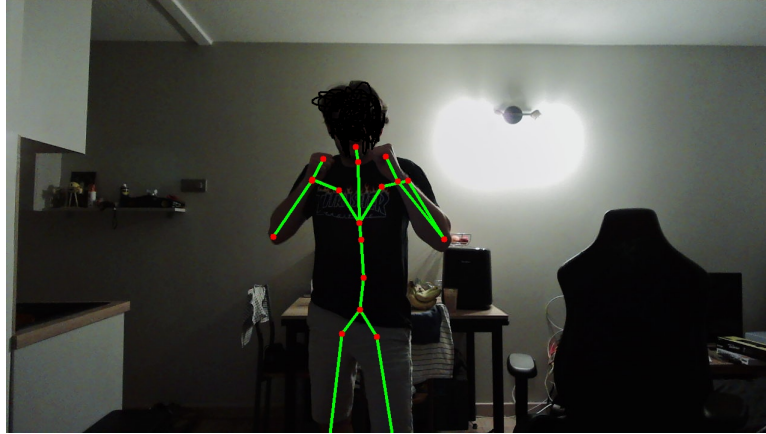


Figure 1: Squelette 2D estimé par NLF sur une image de la webcam (1280×720), sujet en garde de boxe.

4.4 Choix de la version de RT-COSMIK (27/09)

Problème – La version préinstallée ne gère pas une seule caméra

Symptôme. Le dossier `config/cam_params` est vide, et la branche préinstallée (bd80b0d) attend des fichiers de calibration aux noms fixes.

Cause. Dans cette version, `load_camera_parameters` charge toujours au moins deux caméras (`c0_params_color.yaml`, `c2_params_color.yaml` et leur pose relative), et le pipeline triangle systématiquement.

Solution. Installation de la branche `main` de Gepetto (064353f) à côté de l'existante (RT-COSMIK-main), pour ne rien casser. Cette version accepte `--cameras 0` (NLF monoculaire), ne demande que les intrinsèques `config/cam_params/intrinsics/camera_0_intrinsics.yaml`, et choisit le moteur YOLO selon le nombre de caméras. Poids réutilisés, moteur batch 1 renommé `yolov10n_b1.engine` avec un fichier `.meta` contenant 1,640 ; ffmpeg installé.

Les réglages modifiés dans `settings.py` sont regroupés dans le tableau 3 (patch `rtcosmik/patches/0002-settings-single-webcam.patch`).

Paramètre	Défaut	Utilisé	Raison
width, height	1280×720	640×480	Bande passante USB/IP (13 contre 3,7 fps)
fs	40	30	Maximum de la webcam
cameras	(0, 2, 4, 6)	(0,)	Une seule webcam
ik_type	mhe	sbs	Aucun solveur à précompiler
human_height/weight	1,80 m, 70 kg	1,73 m, 67 kg	Mise à l'échelle du modèle
no_trial	test	demo_0X	Un dossier de sortie par prise

Table 3: Réglages de RT-COSMIK modifiés.

4.5 Calibration intrinsèque sans damier imprimé (27/09)

Faute d'imprimante (week-end), le damier (10×7 cases, soit 9×6 coins intérieurs) a été affiché en plein écran sur un ordinateur portable, luminosité maximale. Un écran est parfaitement plan, ce

qui convient bien à la calibration. Le script `scripts/calibration/calib_intrinsics.py` capture 25 vues où le damier est détecté, puis estime le modèle sténopé avec distorsion radiale et tangentielle :

$$s \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = K \begin{pmatrix} x_d \\ y_d \\ 1 \end{pmatrix}, \quad K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad D = (k_1, k_2, p_1, p_2, k_3), \quad (1)$$

où (x_d, y_d) sont les coordonnées normalisées après distorsion.

Problème – Erreur de reprojection élevée

Symptôme. Premier calcul : RMS de 4,44 px (objectif < 1), $f_x = 381,6$ et $f_y = 392,3$ (écart de 3 % suspect pour des pixels carrés).

Cause. La fenêtre d'affinement sous-pixel des coins (`cornerSubPix`) était de 11×11 px. En 640×480 , les cases du damier sont petites à l'image : la fenêtre débordait sur les cases voisines et déplaçait les coins.

Solution. Recalcul sur les *mêmes* 25 images avec une fenêtre 5×5 (`scripts/calibration/recalib.py`, qui écarte aussi itérativement les pires vues si nécessaire) : **RMS de 0,67 px**, $f_x \approx f_y$ à 0,4 % près. La première focale était fausse de 23 %. Le script de capture utilise désormais 5×5 .

Intrinsèques (640×480)		Distorsion	
f_x	492,57 px	k_1	−0,0690
f_y	494,71 px	k_2	0,1739
c_x	361,45 px	p_1	−0,0028
c_y	228,56 px	p_2	0,0327
RMS	0,673 px	k_3	−0,0909

Table 4: Calibration retenue (`config/cam_params/intrinsics/camera_0_intrinsics.yaml`). Champ de vision : $2 \arctan(320/f_x) = 66,0^\circ$ en horizontal, $51,8^\circ$ en vertical.

Point à reprendre : le centre optique est décalé de 41 px vers la droite par rapport au centre de l'image, et le coefficient tangentiel p_2 est relativement élevé. Ces deux paramètres sont corrélés, et ce décalage est typique de vues qui ne couvrent pas assez les bords de l'image (en particulier le bord gauche). Une nouvelle capture insistant sur les coins est prévue.

4.6 Premier lancement du pipeline complet (27/09)

Problème – Plantage au chargement de NLF

Symptôme. `RuntimeError: Unknown builtin op: torchvision::nms` dans le processus pipeline ; les processus caméra restent bloqués à la barrière.

Cause. Le modèle NLF (TorchScript) appelle l'opérateur `torchvision::nms`, qui n'est enregistré dans PyTorch que si `torchvision` a été importé. `src/rtcosmik/nlf/nlf.py` ne l'importe pas, et le processus est lancé en mode `spawn`, donc sans hériter des imports du processus parent.

Solution. Ajout de `import torchvision` en tête de `nlf.py` (patch `rtcosmik/patches/0001-nlf-import-torchvision.patch`). **Modification à signaler au LAAS.**

Après correction, le pipeline démarre, calibre le modèle sur le sujet, puis tourne à **22 itérations/s** (boucle de 45 ms, dont 41 ms pour l'estimation de pose et 2,8 ms pour la cinématique inverse). Les messages restants sont sans gravité : conflit de version SciPy/NumPy système, avertissement `'half' is deprecated` d'Ultralytics (masqué par `export YOLO_VERBOSE=False`), et absence

de repère monde (les positions sont exprimées dans le repère de la caméra ; les angles articulaires n'en dépendent pas).

4.7 Visualisation et synchronisation vidéo / CSV (27/09)

La version `main` n'ouvre pas de fenêtre vidéo : la seule visualisation en direct est Meshcat (serveur web sur le port 7000 du conteneur), qui doit être redirigé vers Windows (redirection de port VS Code). Pour obtenir une démonstration lisible, le script `scripts/analysis/overlay_video.py` projette les marqueurs 3D de `markers.csv` sur la vidéo enregistrée à l'aide de K et D , et affiche les angles des coudes.

Problème – Squelette décalé par rapport à la vidéo

Symptôme. Premier essai : aucun squelette dessiné. Deuxième : squelette très en retard (environ 20 s). Prise `demo_02` : compteur CSV de 586 à 2231 pour seulement 473 images dans la vidéo.

Cause. La colonne `Frame_0` compte les images lues par le processus caméra, à environ 30 par seconde. Interprétation : ffmpeg duplique les images pour tenir la cadence demandée, alors que la webcam n'en livre que 6 à 7 réelles via WSL. La vidéo `.mkv`, copie directe du flux de la caméra, ne contient que les images réelles, sur toute la durée de la prise. Les 586 premières valeurs du compteur correspondent aux environ 20 s de chargement de NLF et de calibration du sujet, pendant lesquelles aucune ligne n'est écrite.

Solution. Recaler linéairement le compteur sur la vidéo (compteur 0 = première image, compteur maximal = dernière image), avec un petit décalage réglable `OFFSET`. `overlay_offsets.sh` génère une vidéo par valeur de décalage pour choisir la meilleure à l'œil. Une synchronisation exacte passera par les horodatages (piste pour le livrable 3). Sur `demo_03`, un décalage nul est déjà presque juste, tandis qu'un décalage de +3 images fait précéder la vidéo par le squelette : l'optimum se situe autour d'une image (environ 0,13 s à 7,5 images réelles par seconde). Le décalage accepte donc des valeurs fractionnaires.

5 Scripts développés

Tous les scripts sont versionnés dans le dépôt `pfe-combat-mirror` et se lancent depuis la racine de RT-COSMIK.

Script	Rôle
<code>tests/test_webcam.py</code>	Ouvre la webcam, mesure les fps réels, détection YOLO sur une image
<code>tests/test_skeleton_nlf.py</code>	Photo avec compte à rebours, squelette 2D NLF (24 articulations)
<code>tests/test_yolo_engine.py</code>	Vérifie qu'un moteur TensorRT se charge et détecte une personne
<code>tests/engine_metadata.py</code>	Lit l'en-tête Ultralytics d'un <code>.engine</code> (lot, taille d'image)
<code>calibration/checkerboard_9x6.png</code>	Damier à afficher en plein écran
<code>calibration/calib_intrinsics.py</code>	Capture automatique de vues du damier et calibration
<code>calibration/recalib.py</code>	Recalcul depuis les images enregistrées, rejet des pires vues
<code>analysis/analyze_take.py</code>	Qualité d'une prise : trous, stabilité des segments, sauts, graphes
<code>analysis/overlay_video.py</code>	Vidéo avec squelette superposé et angles des coudes
<code>analysis/overlay_offsets.sh</code>	Une vidéo de superposition par valeur de décalage
<code>tools/collect_from_container.sh</code>	Exporte du conteneur config, modifications, résultats et versions
<code>tools/setup_rtcosmik_main.sh</code>	Installe RT-COSMIK main, applique les patches, copie la calibration, télécharge les modèles

Table 5: Scripts du dépôt (dossier `scripts/`).

6 Configuration de la caméra

Modèle	UGREEN Camera, USB 0c45:2283
Formats	MJPEG, entre autres 1920×1080, 1440×1080, 1280×960, 1280×720, 1024×768 ; 5 à 30 fps
Mode utilisé	MJPEG 640×480, 30 fps demandés (environ 6 à 13 fps réels via WSL)
Exposition	<code>auto_exposure = 3</code> (priorité ouverture), <code>exposure_dynamic_framerate = 0</code>
Accès	<code>usbipd attach --wsl --busid 5-2</code> , puis <code>/dev/video0</code> dans le conteneur
Placement	Face au sujet ; 2,5 à 3 m recommandés (voir section 7)
Calibration	Tableau 4

Table 6: Configuration de la webcam.

7 Premiers résultats : prise `demo_03`

Prise d'environ 80 s en direct : immobilité pour la calibration du modèle, garde, puis enchaînements libres de directs et de crochets (le protocole prévu n'a pas été suivi strictement). Le script `analyze_take.py` donne :

Indicateur	Valeur		Lecture
Lignes / durée / cadence	1225 / 80,5 s / 15,2 Hz		
Valeurs manquantes	0 %		Aucune perte de suivi
<i>Segment</i>	<i>médiane</i>	<i>variation</i>	
Bras D / G	31,9 / 33,6 cm	7,8 / 6,3 %	Stable
Avant-bras D / G	23,6 / 23,9 cm	11,5 / 10,2 %	Moyen (occultations en garde)
Cuisse D	47,3 cm	9,6 %	Stable
Jambe D	48,5 cm	4,1 %	Un peu longue (pieds hors champ)
Distance bassin-caméra	1,61 m ($\sigma = 15,9$ cm)		Trop près
Sauts du poignet > 30 cm	D : 6 (0,5 %), G : 0		Tous pendant des coups

Table 7: Analyse de la prise `demo_03`. La variation est le coefficient de variation de la longueur du segment.

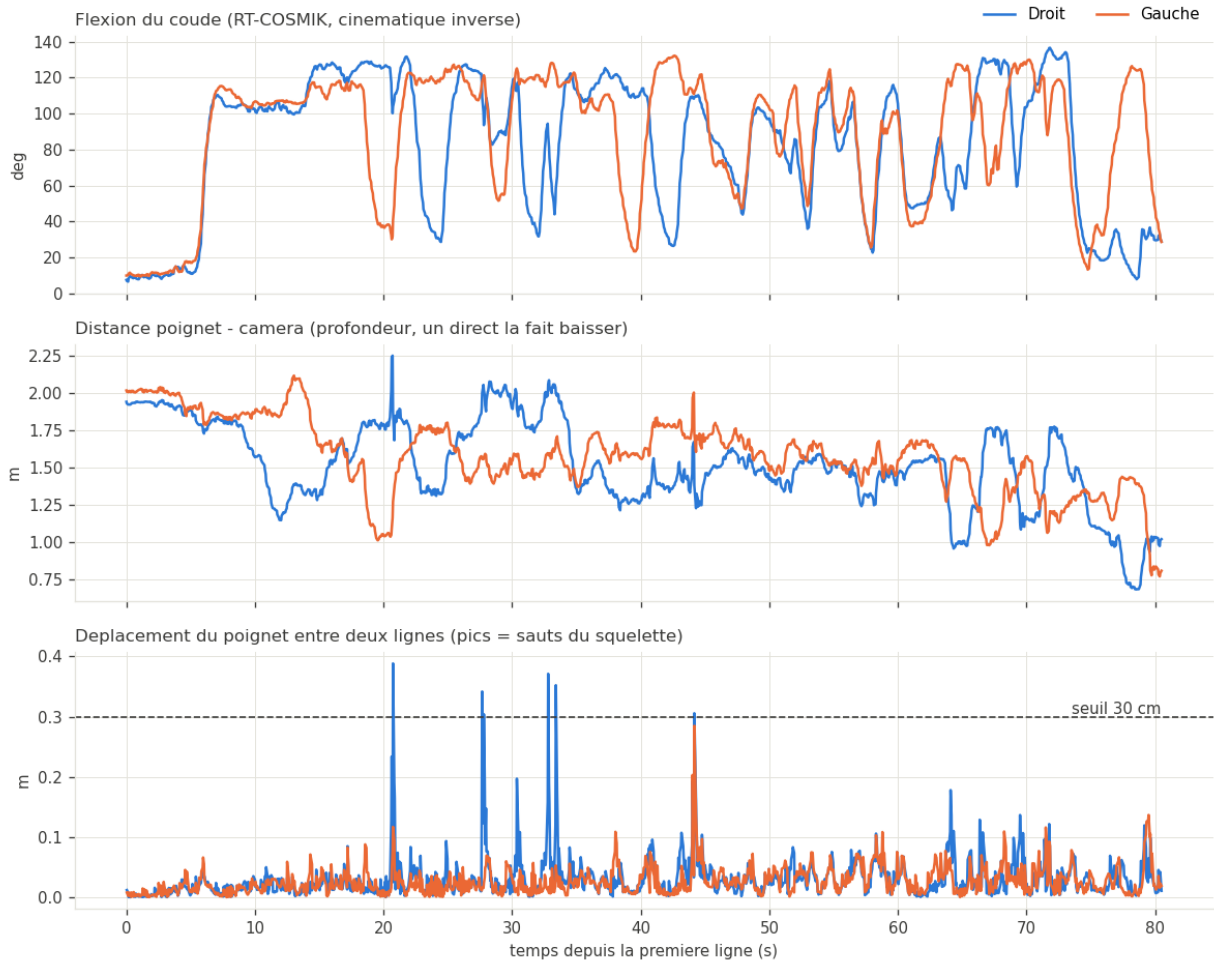


Figure 2: Prise `demo_03`. En haut : flexion des coudes issue de la cinématique inverse (0° = bras tendu). Au milieu : distance poignet-caméra. En bas : déplacement du poignet entre deux lignes successives.

Interprétation.

- Les données ont un sens physique. Coudes à environ 10° pendant la calibration (bras tendus), passage à environ 110° à la mise en garde vers 6 s, puis creux alternés gauche/droite entre 20 et 45 s correspondant aux directs.
- La profondeur suit les coups. À 20,5 s, l'ouverture du coude gauche coïncide avec un rapprochement du poignet gauche de la caméra (de 2 m à 1 m) : l'estimation monoculaire perçoit bien un coup dirigé vers l'objectif.
- Les proportions sont humaines et cohérentes avec un sujet de 1,73 m. Les avant-bras sont les moins stables, ce qui est attendu : ils sont devant le corps et dans l'axe de la caméra.
- Les "sauts" ont tous lieu pendant des coups. Une partie est réelle : un poignet en direct se déplace de plusieurs mètres par seconde, soit 30 cm entre deux images quand on n'a que 6 à 7 images réelles par seconde. C'est un sous-échantillonnage plus qu'une erreur.
- Cadrage : à 1,61 m, la caméra ne couvre que $2 \times 1,61 \times 240/f_y \approx 1,56$ m en hauteur, moins que la taille du sujet. Les pieds sont donc hors champ et les jambes en partie estimées par NLF. À 2,5 m, la couverture passe à 2,43 m.

8 Limites actuelles et suite

Limite	Action prévue
6 à 13 fps réels (USB/IP sous WSL) ; un direct dure 1 à 2 images	Passage sur la machine Debian de Polytech (caméras en <code>/dev/video</code> natif, 30 fps attendus)
Une seule caméra : profondeur et occultations des bras	Deuxième caméra, angles de vue écartés de 45 à 90° (par exemple en haut et sur le côté du cadre du miroir), triangulation et calibration extrinsèque avec <code>cams_calibration</code>
Pas de repère monde	Marqueur ArUco vu par la caméra de référence (<code>cam_to_world</code>)
Centre optique décalé de 41 px	Nouvelle calibration couvrant les bords de l'image
Synchronisation vidéo/CSV approximative	Horodatages communs caméra/capteurs (livrable 3)
Solveur <code>sbs</code>	Évaluer <code>mhe</code> (acados, profil <code>realtime</code>), plus lisse

Table 8: Limites identifiées et actions correctives.

Prochaines étapes.

1. Porter l'installation sur la machine Debian de Polytech (reconstruction de l'image, application des patches, régénération du moteur TensorRT) et mesurer les fps réels.
2. Refaire une prise suivant le protocole (immobilité, garde, 5 directs lents par bras, bras en croix, coups rapides) à 2,5–3 m, bien éclairée.
3. Ajouter la seconde caméra et la calibration extrinsèque.
4. Démarrer le livrable 2 : choix des capteurs de pression, de l'électronique d'acquisition et de leur disposition derrière le miroir.

A Aide-mémoire : lancer une session

```
# Windows (PowerShell admin) : partager la webcam avec WSL
usbipd attach --wsl --busid 5-2
docker start 90034be366af
docker exec -it 90034be366af bash

# Conteneur
v4l2-ctl -d /dev/video0 -c exposure_dynamic_framerate=0
cd ~/workspace/ros_ws/RT-COSMIK-main
export YOLO_VERBOSE=False
python3 scripts/python/core/run_pipeline.py --online --cameras 0
# rester immobile jusqu'à "Model calibration finished", puis q et Ctrl+C

# Analyse de la prise
python3 analyze_take.py output/demo_03
bash overlay_offsets.sh output/demo_03 "0 0.5 1 1.5 2"
```

B Modification apportée au code de RT-COSMIK

```
--- a/src/rtcosmik/nlf/nlf.py
+++ b/src/rtcosmik/nlf/nlf.py
@@ -1,5 +1,6 @@
```

```
import numpy as np
import torch
+import torchvision # charge torchvision::nms, requis par le modele NLF (TorchScript)
from ultralytics import YOLO
```